

Autonomous Cyber Defense Loops: A Multi-Agent Framework for Unified Pre- and Post-Deployment Security Orchestration

Author: Kwesi Asiedu

Institution: Department of Computer & Information Sciences, Towson University

Advisor: Prof. Wassila Lalouani

1. Title

Autonomous Cyber Defense Loops: A Multi-Agent Framework for Unified Pre- and Post-Deployment Security Orchestration

2. Abstract

Modern software security architectures suffer from severe structural fragmentation. While DevSecOps pipelines deploy static analysis tools during build time, Security Operations Centers (SOCs) operate almost exclusively on real-time operational telemetry during runtime. This disconnect creates a critical lag between vulnerability identification, exploit verification, and dynamic policy mitigation. In this paper, we propose the Autonomous Cyber Defense Loop (ACDL), a unified closed-loop framework that connects static security assessment, semantic threat modeling, active exploitation verification, and SOC remediation. By combining Large Language Models (LLMs) with Multi-Agent Reinforcement Learning (MARL), ACDL continuously translates static telemetry (SAST, SCA, Secrets, Container, IaC, and Kubernetes configurations) into semantic attack surface graphs. These graphs direct targeted automated pentesting agents to empirically verify exploitability against active targets (DAST, API, Mobile, and LLM Sandboxes) without human intervention. Verified exploit chains trigger immediate policy updates across SOC platforms and Infrastructure-as-Code definitions. Across a 150-microservice cyber range environment, ACDL achieves a 90.3% reduction in Mean Time to Detect (MTTD), a 97.9% reduction in Mean Time to Remediate (MTTR), and suppresses false-positive alerts by 93.8%.

3. Introduction

The rapid adoption of cloud-native infrastructure, microservice architectures, and continuous deployment pipelines has transformed software engineering. However, security practices have struggled to keep pace with the scale and velocity of contemporary software delivery. Enterprise security architecture relies on isolated tools:

- Static Scanners (Left-Loop): Source code (SAST), dependencies (SCA), embedded secrets, container manifests, Infrastructure-as-Code (IaC), and Kubernetes policies.
- Dynamic Analysis & Active Penetration Testing (Active-Loop): Fuzzers and automated exploitation engines testing active endpoints, web applications, APIs, and mobile binaries.
- Security Operations (SOC-Loop): Real-time monitoring engines, Security Information and Event Management (SIEM) platforms, and Threat Intelligence feeds tracking operational events.

Despite heavy investment in each category, these tools operate in isolated silos. Static scanners generate large volumes of theoretical vulnerabilities, many of which are unreachable in execution or suppressed by existing network mitigations. Conversely, SOC teams respond to runtime anomalies without immediate visibility into the underlying static codebase or configuration flaws that allowed the anomaly to occur. This operational

disconnect leads to severe alert fatigue, delayed remediation cycles, and a persistent window of exposure.

4. Problem Statement

Current cybersecurity tools operate largely as isolated systems, requiring significant human effort to correlate findings, validate vulnerabilities, and coordinate remediation.

Specifically:

1. **Telemetry Disparity:** Static analysis tools generate code-level findings without runtime context, while SOC monitoring platforms generate runtime alerts without repository-level visibility.
 2. **Alert Fatigue:** Security teams are inundated with high volumes of theoretical, unverified static alerts, up to 70% of which are non-exploitable in live environments.
 3. **Remediation Delay:** Translating a confirmed security finding into an actionable Infrastructure-as-Code (IaC) patch or firewall policy update currently requires manual triage, introducing remediation delays averaging days or weeks.
-

5. Purpose of the Study

The primary purpose of this study is to design, implement, and empirically evaluate the Autonomous Cyber Defense Loop (ACDL)—a multi-agent AI-driven framework that bridges the gap between pre-deployment DevSecOps pipelines and post-deployment SOC operations.

This study intends to:

1. Construct a unified multi-modal security telemetry graph that ingests static code defects, container manifests, and infrastructure configurations into semantic threat models.
 2. Automate empirical vulnerability validation using LLM-guided reinforcement learning agents to execute safe, targeted penetration payloads against active dynamic targets.
 3. Establish an autonomous self-healing feedback loop that dynamically commits Infrastructure-as-Code patches and updates SOC firewall policies upon exploit verification.
-

6. Research Questions

To guide this investigation, this dissertation addresses three core research questions:

- RQ1: Can a multi-agent AI system autonomously coordinate security activities across the entire software lifecycle (from static code scanning to dynamic penetration testing and SOC policy generation)?
 - RQ2: Does autonomous multi-agent orchestration significantly reduce the Mean Time to Detect (MTTD) and Mean Time to Remediate (MTTR) vulnerabilities compared to traditional DevSecOps and SOAR workflows?
 - RQ3: Can closed-loop AI verification effectively eliminate false positives and improve overall security posture without introducing breaking changes into continuous integration pipelines?
-

7. Hypotheses

For quantitative evaluation, we establish the following formal hypotheses:

- H1: An autonomous multi-agent cybersecurity framework will reduce vulnerability remediation time (MTTR) by at least 80% compared with conventional manual and rule-based SOAR security workflows.
- H2: Integrating active LLM verification agents with static telemetry graphs will reduce false-positive security alerts by over 85% compared to static analysis alone.

- H3: Automated Infrastructure-as-Code (IaC) patch generation driven by verified attack paths will achieve greater than 90% syntactical and operational deployment accuracy without breaking CI/CD builds.
-

8. Literature Review

8.1 Static Analysis and Infrastructure Security

Static Application Security Testing (SAST) and Software Composition Analysis (SCA) form the foundation of shift-left DevSecOps strategies. Early works focused on Abstract Syntax Tree (AST) parsing and Control Flow Graph (CFG) analysis. Modern static tools extend these principles to container image layers (e.g., Trivy, Gype), infrastructure declaration files (e.g., Checkov, tfsec), and Kubernetes manifests. However, static techniques suffer from high false-positive rates due to the absence of execution context.

8.2 Dynamic Testing and Automated Exploit Generation (AEG)

Dynamic Application Security Testing (DAST) and API fuzzing evaluate application states at runtime by crafting malicious requests and monitoring responses. Automated Exploit Generation (AEG) has evolved from symbolic execution (e.g., Mayhem, Angr) to reinforcement-learning-guided fuzzers (e.g., AFL++, Neuzz). While dynamic methods produce zero false positives upon discovering a working exploit, unguided fuzzing suffers from state-space explosion and low coverage of deep application logic.

8.3 Graph-Based Threat Modeling and Attack Surface Representation

Mapping vulnerability data into structured graphs enables reasoning about multi-stage attacks. Topological threat models, such as Attack Trees and STRIDE matrices, capture potential attacker movements. Recent advances introduce Graph Neural Networks (GNNs) to generate graph embeddings of enterprise infrastructure. However, existing threat graphs remain static snapshots constructed manually or populated strictly from network topology logs.

8.4 Multi-Agent AI and Reinforcement Learning in Security

Deep Reinforcement Learning (DRL) frameworks, such as CybORG and CyberBattleSim, model penetration testing as Markov Decision Processes (MDPs). More recently, Large Language Model (LLM) agents demonstrate capability in semantic reasoning, payload crafting, and automated code refactoring. However, LLM-driven security tools are predominantly evaluated in isolated bench environments rather than integrated end-to-end ecosystems that execute live verification and infrastructure remediation.

8.5 Identification of Research Gaps

Existing solutions operate either strictly pre-deployment (static scanning) or strictly post-deployment (SOC monitoring). Prior literature lacks a closed-loop framework capable of converting static telemetry into targeted dynamic verification exploits and automated IaC remediation.

9. Theoretical & Conceptual Framework

The ACDL framework is grounded in four fundamental theoretical pillars:

1. Multi-Agent Systems (MAS) Theory: Distributed autonomous agents interact, negotiate, and specialize (Left-Loop Scanners, Brain Modelers, Red-Team Attackers, Blue-Team Defenders) to resolve complex global security objectives.
2. Closed-Loop Control Theory: Adapts classic feedback control ($e(t) = r(t) - y(t)$) to software security, where system state errors (vulnerabilities) drive self-healing actuation (IaC patches and firewall rules).

3. Zero Trust Architecture (ZTA): Enforces explicit, continuous verification regardless of network location, treating all unverified static findings as potential untrusted vectors until validated.
4. MITRE ATT&CK; & Cyber Kill Chain: Provides the standardized semantic taxonomy for mapping attacker tactics, techniques, and procedures (TTPs) across multi-hop exploitation chains.

10. Proposed Research Framework / Architecture

The Autonomous Cyber Defense Loop (ACDL) connects 18 security microservices across four operational phases:

Functional Flow Across the 18 Launchpad Applications:

- Code & Infrastructure Ingestion (Phase 1): Static telemetry is gathered simultaneously across 05-SAST, 06-SCA, 07-Secrets, 08-Container, 09-Kubernetes, and 10-IaC Security.
- Semantic Threat Synthesis (Phase 2): Ingested findings are mapped onto the STRIDE Modeler and Attack Chain Visualizer to build a directed threat graph $G = (V, E)$.
- Active Payload Verification (Phase 3): Expedite Strike and AXIOM Exploitability orchestrate multi-agent LLM payloads targeting active 11-DAST, 12-API, 13-Mobile, and AI/LLM Sandbox endpoints.
- Risk Correlation & Hardening (Phase 4): Verified exploit signals populate AI Red Teaming ATT&CK; logs, trigger real-time Aegis Security SOC and Tenable SecurityCenter firewall updates, and execute automated IaC Auto-Patch Engine pull requests.

11. Research Methodology

11.1 Mathematical Formulation of the Threat Surface Graph

We model the enterprise software state as a weighted heterogeneous directed graph:

$$G = (V, E, W)$$

where $V = V_{\text{asset}} \cup V_{\text{vuln}} \cup V_{\text{config}}$, $E \subseteq V \times V$, and $W: E \rightarrow \mathbb{R}^+$ represents accessibility weights derived from CVSS ratings and network adjacency matrices.

11.2 Multi-Agent Markov Decision Process (MAMDP)

Phase 3 active verification is formulated as a Multi-Agent Markov Decision Process:

$$\mathcal{M} = \left(\mathcal{S}, \mathcal{A}_1, \dots, \mathcal{A}_N, \mathcal{P}, \mathcal{R}_1, \dots, \mathcal{R}_N, \gamma \right)$$

where agent reward functions are defined as:

$$\mathcal{R}(s, a) = \alpha \cdot \mathbf{1}_{\text{verified_exploit}} + \beta \cdot \Delta \text{Coverage} - \lambda \cdot \text{Latency}$$

11.3 Self-Healing Execution Algorithm

```
def autonomous_cyber_defense_loop(repository_repo, runtime_target):  
    # Step 1: Static Scanning  
    static_telemetry = run_static_scanners(repository_repo)  
  
    # Step 2: Semantic Threat Graph Synthesis  
    threat_graph = build_threat_graph(static_telemetry)  
    attack_chains = extract_critical_paths(threat_graph)  
  
    # Step 3: Multi-Agent Active Verification  
    verified_exploit_paths = []  
    for path in attack_chains:  
        payload = red_team_agent.generate_payload(path)  
        response = active_executor.dispatch(target=runtime_target, payload=payload)  
        if verify_exploit_success(response):  
            verified_exploit_paths.append((path, payload, response))  
  
    # Step 4: SOC & IaC Auto-remediation  
    for path, payload, response in verified_exploit_paths:  
        aegis_soc.update_ids_rules(signature=payload.signature)  
        tenable_sc.ingest_verified_finding(path)  
        iac_patch = generate_terraform_patch(path.root_cause)  
        apply_iac_pull_request(iac_patch)  
  
    return len(verified_exploit_paths)
```

12. Experimental Design

12.1 Experimental Environment & Setup

Experiments were conducted inside an enterprise cyber range consisting of a 64-node Kubernetes cluster running 150 microservices with injected vulnerability vectors (synthetic and real-world CVEs).

12.2 Comparative Treatments

We evaluate three distinct security workflows:

1. Baseline Treatment A: Conventional Manual DevSecOps Pipeline (Static scanning + manual triage + manual ticketing).
2. Baseline Treatment B: Rule-Based SOAR Automation (Linear rule scripts connecting SIEM alerts to standard playbooks).
3. Proposed Treatment C: Autonomous Cyber Defense Loop (ACDL) Framework.

12.3 Evaluation Variables

- Independent Variable: Security orchestration paradigm (Manual vs. SOAR vs. ACDL).
- Dependent Variables: Mean Time to Detect (MTTD), Mean Time to Remediate (MTTR), False Positive Reduction Rate (FPRR), Patch Accuracy (PA), Attack Chain Coverage (ACC), and Human Intervention Minutes (HIM).

13. Data Collection and Analysis

13.1 Data Collection Protocol

Log telemetry was gathered automatically across 500 execution runs per treatment:

- Time-stamped event logs recorded from code commit to pull request merge.

- HTTP request/response PCAP dumps recorded during Phase 3 verification.
- Synthetic vulnerability injection truth tables tracking actual vs. reported exploits.

13.2 Statistical Analysis Methods

- Analysis of Variance (ANOVA): Used to test for statistically significant differences in MTTD and MTTR across treatments ($p < 0.001$).
- Tukey's Honest Significant Difference (HSD) Post-Hoc Test: Conducted to confirm pairwise significance.
- Confusion Matrix & Precision/Recall Metrics: Applied to measure false-positive suppression accuracy.

14. Results

14.1 Quantitative Performance Findings

Metric	Treatment A (Manual DevSecOps)	Treatment B (Rule-Based SOAR)	Treatment C (Proposed ACDL)	Statistical Significance (p-value)
Mean Time to Detect (MTTD)	4.2 hours	1.1 hours	6.4 minutes	$p < 0.001$
Mean Time to Remediate (MTTR)	72.5 hours	14.8 hours	18.2 minutes	$p < 0.001$
False Positive Alert Ratio	68.4%	42.1%	4.2%	$p < 0.001$
IaC Patch Accuracy (PA)	N/A (Manual)	61.2%	94.8%	$p < 0.001$
Critical Path Coverage (ACC)	31.0%	54.5%	98.2%	$p < 0.001$
Human Intervention (HIM/incident)	145 mins	35 mins	2.1 mins	$p < 0.001$

15. Discussion

15.1 Hypothesis Testing & RQ Findings

- RQ1 & H1 Confirmed: The ACDL framework autonomously coordinated activities across static scanning, dynamic pentesting, and SOC hardening. MTTR dropped from 72.5 hours to 18.2 minutes (97.9% reduction), fully supporting H1.
- RQ2 & H2 Confirmed: Active multi-agent payload validation suppressed false positives from 68.4% down to 4.2% (93.8% noise reduction), confirming H2.
- RQ3 & H3 Confirmed: Auto-generated Terraform patches achieved 94.8% patch accuracy without breaking CI/CD builds, confirming H3.

15.2 Operational Significance

By unifying static and operational telemetry into a closed loop, ACDL eliminates the artificial boundary between pre-deployment security engineering and post-deployment operations, solving the industry-wide challenge of alert fatigue.

16. Research Contributions

This dissertation delivers five groundbreaking contributions to the field of computer science and cybersecurity:

1. Novel ACDL Architecture: The first unified closed-loop framework connecting static code analysis (SAST/SCA/IaC) directly to active dynamic verification and SOC policy generation.
 2. Multi-Agent Exploit Verification Protocol: An LLM-guided reinforcement learning algorithm that safely crafts and executes targeted verification payloads to eliminate static false positives without human intervention.
 3. Multi-Modal Threat Surface Graph Representation: A heterogeneous graph model integrating code-level defects, infrastructure misconfigurations, and network reachability into semantic attack paths.
 4. Automated Self-Healing IaC Remediation: A dynamic patch engine capable of auto-generating, syntax-checking, and committing Terraform/Kubernetes remediation code in response to verified runtime exploits.
 5. Reusable 18-App Cyber Range Testbed: A standardized experimental benchmark environment for evaluating autonomous cyber defense systems.
-

17. Limitations

1. LLM Non-Determinism: Generative AI models occasionally output variations in payload syntax, requiring Abstract Syntax Tree (AST) validation gates before execution.
 2. Dynamic Testing Throttling: Active exploit verification generates HTTP traffic and container CPU usage, requiring throttling mechanisms during peak production hours.
 3. Legacy Codebases: Systems lacking automated Infrastructure-as-Code definitions (e.g., unmanaged physical servers) cannot utilize the automated IaC self-healing patch module.
-

18. Future Research

1. Adversarial Co-Evolutionary Cyber Ranges: Extending the Red-Team and Blue-Team agent networks into a zero-sum game environment where attackers and defenders continuously evolve zero-day capabilities.
 2. Federated Security Knowledge Networks: Applying privacy-preserving federated graph learning across multiple enterprises to share threat intelligence without disclosing proprietary source code.
 3. Formal Verification of Auto-Patches: Integrating formal methods and SMT solvers to mathematically guarantee that auto-generated infrastructure patches introduce zero functional regressions.
-

19. Conclusion

This research demonstrates that cybersecurity orchestration can transcend disconnected, manual toolchains through autonomous closed-loop AI architecture. The Autonomous Cyber Defense Loop (ACDL) unifies 18 distinct security applications across the software lifecycle. By combining static telemetry graphs, multi-agent reinforcement learning, and automated IaC remediation, ACDL achieves a 97.9% reduction in remediation time while suppressing false-positive alert noise by over 93%. This work establishes a foundation for self-healing software ecosystems.

20. References

1. Landauer, M., et al. (2023). "Deep Reinforcement Learning for Cyber Defense: A Survey." IEEE Transactions on Information Forensics and Security.

2. Zhou, Y., et al. (2022). "DevSecOps: A Systematic Literature Review." ACM Computing Surveys, 55(4), 1-36.
 3. Shan, C., et al. (2024). "Large Language Models in Autonomous Penetration Testing." Proceedings of the IEEE Symposium on Security and Privacy (S&P);).
 4. MITRE Corporation. (2024). "MITRE ATT&CK; Matrix for Enterprise." Available: <https://attack.mitre.org/>
 5. NIST. (2023). "Zero Trust Architecture." NIST Special Publication 800-207.
 6. Avgerinos, T., et al. (2014). "Automatic Exploit Generation." Communications of the ACM, 57(2), 74-84.
 7. Scarfone, K., & Mell, P. (2021). "Technical Guide to Information Security Testing and Assessment." NIST Special Publication 800-115.
 8. Vaswani, A., et al. (2017). "Attention Is All You Need." Advances in Neural Information Processing Systems (NeurIPS).
-

21. Appendices

Appendix A: Launchpad 18-App Security Node Mapping Table

- 05-SAST, 06-SCA, 07-Secrets, 08-Container, 09-Kubernetes, 10-IaC Security, STRIDE Modeler, Attack Chain Visualizer, Expedite Strike, AXIOM Exploitability, 11-DAST, 12-API Security, 13-Mobile Security, AI/LLM Sandbox, AI Red Teaming, Aegis Security SOC, Tenable SecurityCenter, IaC Auto-Patch Engine.

Appendix B: Red-Team Agent Verification Prompt Template

```
agent_role: Red-Team Active Exploit Verifier
inputs:
  target_url: "{target_endpoint}"
  attack_vector: "{attack_chain_path}"
  vulnerability_type: "{stride_category}"
instructions:
  1. Construct minimal non-destructive HTTP verification payload.
  2. Dispatch payload via dynamic proxy interface.
  3. Analyze HTTP status code, headers, and body reflection.
  4. Return JSON response with boolean 'verified' key and proof evidence.
```